Circuit Breaker Pattern

Category: Resilience **One-liner:** Wrap calls to a remote dependency in a component that stops calling it once it's clearly failing — failing instantly instead of hanging, and probing periodically to see whether it recovered.

Origin: Popularised by **Michael Nygard** in his book *Release It!*

Part 0 — Concepts you need first

What is a thread, and why do we care?

A **thread** is one line of execution inside a program. A typical web server keeps a **thread pool** — a fixed set of threads, say 200 — and assigns one to each incoming request. When the request finishes, the thread returns to the pool for reuse.

The critical property: a thread waiting on a network call is doing nothing else. It's blocked, occupying a slot, until it gets a response or gives up.

This single fact explains why the Circuit Breaker exists. If you have 200 threads and all 200 are blocked waiting on a slow service, your server can accept no requests at all — for *anything*. It looks completely dead, even though its own code is perfect.

What is a timeout?

The maximum time you'll wait for a response before giving up. If you call a service with a 30-second timeout and it never responds, your thread is stuck for the full 30 seconds.

Timeouts are necessary but not sufficient. They stop you waiting *forever*, but if a service is down and you keep calling it, every request still burns its full timeout before failing. A hundred requests a second against a dead service with a 30-second timeout means three thousand threads tied up. Timeouts bound each individual wait; the circuit breaker prevents the wait from happening at all.

What is a "cascading failure"?

When the failure of one component causes the failure of others that depend on it, spreading through the system like falling dominoes. The key insight is that **the components that fail second were never broken** — they were killed by waiting on something that was.

What is a fallback?

A substitute response used when the real one is unavailable — cached data from earlier, a sensible default, or a degraded version of the feature. It's how you turn a hard failure into a mild one.

What does "fail fast" mean?

Returning an error *immediately* rather than after a long wait. Counter-intuitively this is often better for users: two seconds of waiting followed by an error is worse than an instant, clear "this feature is unavailable right now."

Part 1 — The metaphor

An **electrical circuit breaker** in your home monitors current. When there's a dangerous overload it **trips**, cutting the circuit to prevent fire or damage to appliances. Later you flip it back on. If the fault is still there, it trips again immediately.

Two features carry over precisely: 1. **It cuts the connection to prevent wider damage** – better to lose one circuit than the house. 2. **Restoration is a deliberate test**, not automatic – you flip it back and see what happens.

A software circuit breaker trips to stop calls to a failing service, letting the system **fail fast and degrade gracefully** rather than grinding to a halt.

Part 2 – The problem

Distributed applications constantly call remote things – other microservices, databases, external APIs. Every such call risks failure or a slow response. And if a service becomes unresponsive while the application keeps calling it, the results are disastrous: **requests pile up, threads hang, and the failure cascades into a wider outage.**

Cascading failures, step by step

Service A depends on Service B. B goes down, or – worse – becomes very slow.

1. Calls from A to B **hang until they time out** 2. During each wait, A's resources – threads, memory – are **tied up doing nothing** 3. This happens frequently, and across many threads simultaneously 4. A becomes overwhelmed – it **runs out of threads or memory** 5. **A now fails**

> Read step 5 carefully: **A is broken even though nothing is wrong with A's own code.** Its own logic is fine, its own database is fine. It died of waiting. That's the cascading failure, and preventing it is the entire purpose of this pattern.

And it doesn't stop at A. Whatever calls A now waits on A, and the same thing happens to them. One slow service at the bottom can take down everything above it.

> **Slow is worse than down.** If B refuses connections instantly, A gets an immediate error and moves on. If B accepts connections and then takes 30 seconds, A's threads are held hostage. Counter- intuitive, but a service that's merely slow does far more damage than one that's cleanly dead.

Unresponsive services and wasted work

Waiting on slow services with long timeouts means users experience delays and threads stay blocked.

Blindly retrying a known-failing operation wastes CPU cycles and **overloads the failing service even more**. The course's example: if dozens of components keep retrying against a database that's already struggling, it just makes things worse.

> This is the vicious circle: the service is slow because it's overloaded → clients time out and retry → the retries add *more* load → it gets slower → more timeouts and retries. Without intervention a service can be prevented from ever recovering, purely by the traffic of clients trying to reach it.

When a fault is unlikely to resolve quickly – a network outage, a crashed service – **continuously attempting the operation is pointless**. It doesn't solve the problem, and it prevents the application from handling the failure gracefully.

What's needed: a way to detect failure quickly and stop sending traffic to the failing component *before* it drags down the entire system.

Part 3 – The solution

Wrap calls to an external service in a **Circuit Breaker** component that monitors the outcomes of those calls, and stops making them once failures cross a threshold.

Two things happen when it trips, and the second is easy to overlook:

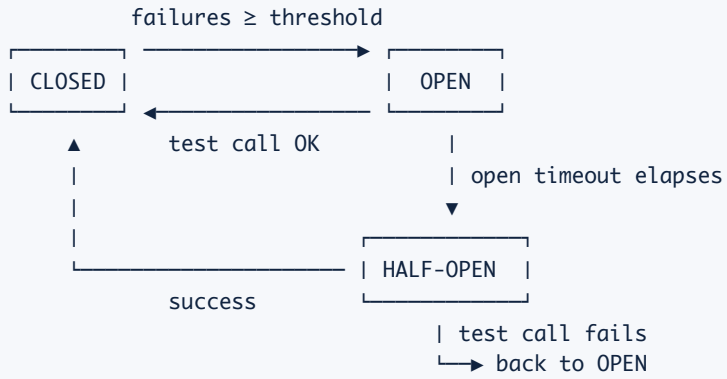
1. Your application stops waiting. Calls are rejected immediately, so you preserve your own responsiveness and resources. Threads stay free for work that can actually succeed.

2. The failing service stops receiving load. An open circuit gives the struggling service **a chance to actually recover** – breaking the vicious circle above.

The pattern prefers **fast failure and graceful degradation** over long waits and resource exhaustion.

Part 4 – The three states

This is the core of the pattern, and what interviewers ask about.



> *Naming warning – this trips people up constantly:* **CLOSED means traffic flows** and **OPEN means traffic is blocked.** It feels backwards until you remember it's an electrical circuit: a closed circuit is a complete loop that current flows through; an open circuit is broken, so nothing flows.

CLOSED – normal operation

The circuit is closed, so traffic flows to the service as usual. The breaker **monitors the number of recent failures**. If failures exceed a configured **failure threshold** within a certain time window, it transitions to Open.

While closed, **successful calls generally reset the failure counter**, so an occasional transient error won't trip the breaker immediately. This matters: you want to distinguish "one blip" from "this service is genuinely broken."

OPEN – tripped

The breaker has detected too many failures and opened the circuit. Calls are **blocked immediately** – the application gets an error or fallback response **without any attempt at the real operation**.

Open state lasts for a configured **timeout period** (the open interval). The purpose is twofold: avoid useless load, and give the troubled service time to recover.

HALF-OPEN – trial mode

After the open timeout expires, the breaker allows **a limited number of test requests** through to the real service. The course describes it as cautiously probing: *"Are you OK now?"*

- **A test request succeeds** → the service is likely back; the breaker resets to **Closed** (usually resetting failure counters)
- **A test request fails** → the breaker flips **immediately back to Open**, and the timeout period starts over

Only a few requests are let through in half-open – this is deliberate, **to avoid blasting a recovering service with traffic**. Often one success is enough to close the circuit; some implementations require several consecutive successes to be sure.

> Half-open is the cleverest part of the design. Without it you'd need either a human to reset the breaker, or a blind reopening of full traffic that would instantly re-kill a service that's only just getting back on its feet.

The per-call logic

Every call through the breaker performs a quick state check:

- **Closed** → attempt the call and record the outcome
- **Open** → skip the call and immediately return a failure or fallback
- **Half-Open** → allow the call (perhaps limiting concurrency), then adjust state based on the result

The breaker acts as a **safety valve**: it ensures a failing component doesn't drag down the rest.

Part 5 – Integration: it's additive, not invasive

Adding a circuit breaker **does not require a system overhaul**. It's a protective layer wrapped around existing service calls, leaving the underlying service logic untouched.

Every request is made *through* the breaker, and the breaker decides whether to forward it based on current state and recent outcomes.

Practical rollout advice from the course: introduce it gradually. Start with **critical services whose failure would significantly impact the system**, then expand to protect other services as the benefits become evident.

Part 6 – Configuration

Parameter	What it controls
Failure Threshold	The limit of failed requests before the breaker switches from Closed to Open
Timeout Period	How long the breaker stays Open before moving to Half-Open – the recovery time given to the service
Number of Requests in Half-Open	How many requests are allowed through while testing whether the service recovered

These values must **strike a balance** between protecting the service, allowing it time to recover, and keeping it as available as possible for handling requests.

The right settings depend on:

- The nature of the service and the system
- The expected load on the service
- How critical the service is to overall functionality

These parameters shouldn't be static

The course makes a point worth remembering: they **can and should be adjusted dynamically** in response to system state and performance. In times of high demand you might:

- **Increase the failure threshold** – let the service endure more failed requests before tripping
- **Reduce the timeout period** – attempt recovery sooner, so the service can handle more requests during peak times

Part 7 – Worked example (Java)

A basic implementation with no external libraries, to illustrate the core algorithm.

```
public class CircuitBreaker {
    private enum State { CLOSED, OPEN, HALF_OPEN }

    private State state = State.CLOSED;
    private int failureCount = 0;
    private final int failureThreshold;
    private final long openTimeout;      // how long to stay open before half-open
    private long lastFailureTime = 0;

    public CircuitBreaker(int failureThreshold, long openTimeout) {
        this.failureThreshold = failureThreshold;
        this.openTimeout = openTimeout;
    }

    public synchronized <T> T call(Callable<T> remoteCall) throws Exception {

        // 1. If OPEN, has the timeout passed?
        if (state == State.OPEN) {
            long timeSinceFailure = System.currentTimeMillis() - lastFailureTime;
            if (timeSinceFailure < openTimeout) {
                // Still within the open window - fail fast, don't call the service
                throw new IllegalStateException(
                    "Circuit is OPEN. Failing fast without calling remote service.");
            } else {
                // Timeout has passed - allow one trial call
                state = State.HALF_OPEN;
            }
        }

        // 2. Attempt the call (allowed when CLOSED or HALF_OPEN)
        try {
            T result = remoteCall.call();
            // Success: reset the counter and close the circuit
            failureCount = 0;
            state = State.CLOSED;
            return result;
        } catch (Exception e) {
            // Failure: count it and record when it happened
            failureCount++;
            lastFailureTime = System.currentTimeMillis();
            if (failureCount >= failureThreshold) {
                state = State.OPEN;   // trip
            }
            throw e;   // propagate (or return a fallback here)
        }
    }
}
```

Walking through the mechanics

The state check comes first. Before doing anything else the breaker asks whether it's Open. If so, and the timeout hasn't elapsed, it throws immediately – **the remote service is never contacted**. That's the whole point: no network call, no thread blocked, microseconds instead of seconds.

`Callable<T>` **makes it generic.** `Callable` is a Java interface for "some operation that returns a value and may throw." Accepting one means this breaker can wrap *any* operation — an HTTP call, a database query, anything — rather than being tied to one specific service.

`synchronized` **handles concurrency.** Many threads may call this simultaneously. Without synchronization two threads could both read `failureCount` as 2, both increment it, and you'd lose a count. *(This is also the example's main performance weakness — see below.)*

Success resets everything. Note that success sets `state = CLOSED` unconditionally. From Half-Open, that's the recovery path. From Closed, it stays closed and clears accumulated failures.

Failure records the time. `lastFailureTime` is what the Open-state check measures against to know when the timeout has elapsed.

Using it

```
CircuitBreaker cb = new CircuitBreaker(3, 10000);  // trip after 3 failures, stay open 10s

try {
    String data = cb.call(() -> unreliableService.getData());
    // use data...
} catch (Exception e) {
    useCachedData();   // fallback
}
```

What happens in sequence: 1. `getData()` fails three times in a row → the circuit opens 2. Any attempt within the next 10 seconds throws **immediately**, without calling `getData()` at all 3. After 10 seconds the next call is allowed through (Half-Open) 4. If it succeeds → circuit closes, normal operation resumes 5. If it fails → circuit re-opens and blocks for another 10 seconds

What this example simplifies

The course notes production breakers are more sophisticated. Specifically:

- **A naive counter, not a rolling window.** Here, three failures trip the breaker *whenever* they happen — even three failures spread across an hour of otherwise perfect traffic. Real implementations use a **rolling time window** ("more than 50% of calls in the last 10 seconds failed"), which is far more meaningful.
- `synchronized` **serialises every call.** Every request through this breaker acquires the same lock, so it becomes a bottleneck under load. Production libraries use **atomic operations and non-blocking synchronization** instead.
- **All exceptions count as failures.** A validation error from the remote service would trip the breaker even though the service is perfectly healthy. Real breakers distinguish which exceptions indicate the *service* is unwell.
- **No fallback built in.** The breaker throws and the caller handles it. Most frameworks let you register a fallback function directly.

Part 8 — Performance implications

The breaker adds a small overhead per call — checking state and updating counters. **This overhead is negligible** compared to the alternative.

The comparison that matters: without a breaker, a failing call ties up a thread for **several seconds** until timeout. With one, the failure is handled in **milliseconds**.

> A tiny check is a small price to pay for avoiding a meltdown.

For extremely performance-sensitive scenarios there are options like asynchronous calls or optimised breaker logic, but generally the benefits far outweigh the overhead.

Tuning the failure threshold

- **Too low** → the circuit trips on every minor glitch, causing unnecessary interruptions and **flapping** (rapid on/off cycling)
- **Too high** → the breaker waits too long to trip, failing to protect the system until a lot of errors have already occurred

The course's calibration: tripping after **1** failure is too sensitive; tripping after **100** is too late. Pick a threshold that **distinguishes normal transient failures from a real problem**.

Tuning the open timeout

- **Too short** → the breaker goes Half-Open and tests the service **before it has recovered**, causing another immediate failure and creating noisy open → half-open → open **flapping**
- **Too long** → your application refuses calls longer than necessary, degrading user experience even after the dependency is healthy again

The timeout should roughly correspond to **how long you expect the service might need to recover**.

Finding the right values

This requires monitoring and tweaking. Good practice: monitor **how often the breaker opens, how long it stays open, and whether it's tripping too frequently.** Metrics like failure counts, open events, and half-open trial outcomes should feed dashboards. With that data you adjust.

In a high-traffic system, consider a **percentage-based threshold** rather than an absolute count, so occasional spikes don't trip it.

Part 9 – Best practices and trade-offs

Fallbacks. When a breaker opens, have a fallback strategy rather than just returning an error – cached data, a default value, or redirecting to a simpler alternative. This is what makes the system **degrade gracefully instead of failing completely**. Many frameworks let you specify a fallback function to execute when open.

Combine with retries and timeouts. The Circuit Breaker doesn't replace other resilience patterns – it works alongside them. You still implement **timeouts** for calls, and may **retry** a few times on certain failures before counting one as a failure. They work in tandem: retry a couple of times in case it's a transient network glitch, but let the breaker stop you trying after repeated failures that indicate a real outage. **Bulkheads** and rate limiting complement both.

Monitoring and alerts. A tripped breaker usually means something is wrong with a downstream service, so **set up alerts**. Have the breaker log or emit an event whenever it opens or closes; your monitoring can catch these and alert on-call engineers or trigger automated recovery.

> Without this, the pattern has a hidden danger: the system quietly protects itself, users see a degraded feature, and **nobody realises anything is broken** because there's no flood of errors. The breaker suppresses exactly the signal that would normally raise the alarm.

Thread-safety and concurrency. In multi-threaded server applications, internal counters and state transitions **must be thread-safe** – you don't want two threads concurrently deciding they're both "first" to open the circuit. Use atomic counters, synchronized sections, or a library. High-performance libraries use non-blocking synchronization and atomic operations to minimise overhead.

Avoiding misuse. Apply breakers to calls that are **potentially unreliable or high-latency**, like network calls. There's **no benefit** on in-memory operations that are very unlikely to fail. Be cautious in systems where failures are very localised or quickly self-correcting – **an aggressive breaker constantly tripping on ephemeral errors does more harm than good.** The pattern must be tuned to your system's reliability needs.

Part 10 – Use cases

Microservices architecture

An e-commerce application split into user management, product catalog, shopping cart, and order processing. Order processing relies on the product catalog to validate availability and user management to validate credentials.

If the **product catalog starts failing**, without safeguards those failures propagate into order processing, degrading the user experience or causing a total outage. With a breaker in front of the catalog, order processing detects failures quickly, stops sending requests, and falls back to a **cached product list** – or fails gracefully with a relevant message.

External API integration

External APIs are **outside your control** and can be unpredictable: downtime, latency spikes, rate limiting policies.

A weather forecasting application pulling from several external weather APIs puts a breaker on **each one**. If one starts failing or becomes slow, the breaker isolates that specific API so its problems don't affect overall performance. The application can then **switch to another API** or provide degraded service until the faulty one recovers.

Database access

Database issues – network problems, resource contention, server failures – cause slow responses or errors.

- **Reads:** in a read-heavy system, a breaker monitors database health. On detecting an increasing error rate or latency, it trips and **redirects reads to a read replica or a cache**, keeping the service available
- **Writes:** the same can apply, but **with more caution**. On increased errors the breaker could trip and temporarily **buffer write operations** – but this must be handled carefully, as **data consistency is at risk** and the buffer itself could become a bottleneck

---

Part 11 – Real-world examples

Netflix and Hystrix. Netflix famously implemented this pattern through their open-source library **Hystrix**. In their microservices web, if a backend service becomes slow or fails, Hystrix isolates that failing endpoint, stopping the failure cascading and returning fallback responses so the user experience is only minimally affected.

> The canonical illustration: **if the recommendations service is down, Netflix can still stream video.** The app omits the recommendations row or shows a default message, rather than failing entirely. That's the whole pattern in one sentence – a non-essential dependency failing must not take down the essential function.

E-commerce product pages. A large site with services for product info, inventory, pricing, and reviews. If the **reviews service** starts failing, a breaker on those calls trips so the product page **skips reviews** rather than slowing the entire page. Product details and pricing load normally, and the page shows something like "Reviews are unavailable at the moment" – **graceful degradation**. Without the breaker, the page might hang or crash trying endlessly to fetch reviews. Similar strategies apply to payment gateways and shipping calculators: if the dependency is down, fall back quickly ("Sorry, we can't process payments right now") instead of making users wait and wonder.

Cloud and service meshes. Many cloud architectures incorporate breakers to improve stability when calling remote services, and service mesh technologies provide breaker mechanisms **at the infrastructure level**, automatically preventing one overloaded microservice from overloading others. In an **API Gateway** you might

configure breaker rules per backend route, so an erroring backend gets rejected for a short period — protecting the backend and freeing the gateway for other routes.

Enterprise integration. Even outside microservices, any integration with an external system — a SOAP or REST API, a database, a messaging system — benefits. A middleware system calling a slow legacy SOAP service uses a breaker to **avoid tying up all its worker threads** on that one slow dependency. The pattern also appears in reactive programming and stream processing for **backpressure**: if a downstream consumer is too slow or failing, the upstream producer trips a breaker and stops feeding it until it recovers.

Libraries: Hystrix (Java, now in maintenance mode), **Resilience4j** (Java), **Polly** (.NET), plus built-in support in cloud platforms and service meshes.

Part 12 — System design example: distributed social media platform

Services: **User Management**, **Post Management**, **Feed Generation**, and more — each running on multiple nodes for high availability and load balancing.

Breakers apply in several places:

In front of User Management. Services relying on it — Post Management, Feed Generation — quickly detect when it's struggling. They then **reduce the load on it** by providing degraded service: displaying **cached user information** or **simplified post feeds**.

Between Post Management and its database. If the database experiences problems, the breaker trips and Post Management starts **serving cached posts**, or **temporarily stops accepting new posts**.

In front of the external email service used for notifications. If it starts failing, the breaker prevents the notification feature from impacting the rest of the system.

Distributed breaker state

An interesting design consideration: use a **shared circuit breaker per service, stored in a distributed cache**. This maintains consistency across all nodes — if a breaker trips on one node, it trips on all of them.

This must be **balanced against the additional complexity and potential performance impact** of consulting a distributed cache on every call.

> The trade-off in plain terms: with per-node breakers, each node must independently discover the service is broken, so each burns its own set of failures learning the same lesson. With shared state, one node's discovery protects everyone — but now every call depends on a cache lookup, and the cache becomes something else that can fail.

Monitoring

Complement the breakers with **comprehensive monitoring and alerting** — monitor the state of all breakers and alert developers or administrators when one trips, ensuring quick detection and remediation.

> **The goal is not to eliminate failures** — they're inevitable in any system. The objective is to **manage failures effectively**, preventing them from cascading into system-wide outages.

Part 13 — Common beginner misconceptions

"Open means working." The most common mix-up. Open = blocked, Closed = flowing. Think electrical circuits.

"The circuit breaker retries for you." No. It *stops* calls. Retry is a separate pattern; they're used together but do opposite things — retry tries harder, the breaker gives up sooner.

"It fixes the broken service." It doesn't. It protects the *caller* and gives the failing service breathing room. Someone still has to fix the actual problem — which is why alerting matters.

"One breaker per application." No — one breaker **per dependency**. If your service calls three others, you need three independent breakers. A shared one would mean a failure in the reviews service blocks your calls to payments.

"If I have timeouts I don't need this." Timeouts bound each individual wait; the breaker prevents the waiting from happening at all once the service is known bad. You need both.

"It makes my system slower." The state check is microseconds. Compared to a 30-second timeout on every call to a dead service, it makes failure handling thousands of times faster.

Part 14 — Check your understanding

1. Why does CLOSED mean traffic flows? 2. Why is a slow service more dangerous than a completely dead one? 3. What two separate benefits does an open circuit provide? 4. Why does Half-Open allow only a *few* requests through? 5. What's the hidden risk of a circuit breaker working well? 6. Why does the example's `synchronized` become a problem at scale?

(Answers: 1 — it's an electrical metaphor; a closed circuit is a complete loop that current flows through; 2 — a dead service refuses instantly so callers move on, while a slow one holds caller threads hostage until timeout; 3 — the caller stops wasting resources waiting, and the failing service stops receiving load so it can recover; 4 — to avoid overwhelming a service that has only just started recovering; 5 — it suppresses the errors that would normally alert you, so problems go unnoticed without explicit alerting on breaker state; 6 — every call acquires the same lock, so the breaker itself becomes a bottleneck under high concurrency.)

Part 15 — Interview soundbite (30 seconds)

> "A circuit breaker stops you repeatedly calling a dependency that's already failing. The problem is cascading failure: if Service B hangs, Service A's threads block waiting on it, and A runs out of threads and dies even though A's own code is fine. Slow is actually worse than down, because a dead service refuses instantly while a slow one holds your threads hostage. The breaker wraps the call and has three states — Closed, where traffic flows and failures are counted; Open, once failures cross a threshold, where calls are rejected instantly with a fallback; and Half-Open, after a timeout, where a few trial requests probe whether it recovered. Success closes it, failure re-opens it, and it's only a few requests so you don't re-kill a service that's just getting back up. It does two things at once: your service stops wasting resources waiting, and the failing service stops receiving load so it can actually recover — that second part breaks the vicious circle where retry traffic prevents recovery. Netflix's Hystrix is the classic example: if recommendations is down they still stream video and just drop that row. Tuning matters — too sensitive and it flaps, too lenient and it doesn't protect you, so you generally want a percentage-based threshold over a rolling window rather than a raw count. And you must alert on breaker state, because a breaker working well hides the errors that would otherwise tell you something's broken."

Key takeaways: prevent cascading failures · fail fast and recover · tune and monitor · use in combination with other patterns.

Part 16 — Implementation

A. Runnable demo — watch the state machine work

Full source: `demos/chapters/06-circuit-breaker/CircuitBreakerDemo.java`

```
cd demos && ./gradlew runCh06
```

```
import java.util.concurrent.Callable;
import java.util.concurrent.atomic.AtomicInteger;

public class CircuitBreakerDemo {

    // ---------- The circuit breaker ----------
    static class CircuitBreaker {
        enum State { CLOSED, OPEN, HALF_OPEN }

        private State state = State.CLOSED;
        private int failureCount = 0;
        private final int failureThreshold;
        private final long openTimeoutMs;
        private long lastFailureTime = 0;
        private final String name;

        CircuitBreaker(String name, int failureThreshold, long openTimeoutMs) {
            this.name = name;
            this.failureThreshold = failureThreshold;
            this.openTimeoutMs = openTimeoutMs;
        }

        synchronized <T> T call(Callable<T> action, Callable<T> fallback) throws Exception {
            if (state == State.OPEN) {
                if (System.currentTimeMillis() - lastFailureTime < openTimeoutMs) {
                    log("REJECTED (circuit OPEN) -> fallback");
                    return fallback.call();          // fail fast, no remote call
                }
                transitionTo(State.HALF_OPEN);       // timeout elapsed: allow a probe
            }

            try {
                T result = action.call();
                if (state == State.HALF_OPEN) {
                    log("probe SUCCEEDED");
                }
                failureCount = 0;
                transitionTo(State.CLOSED);
                return result;
            } catch (Exception e) {
                failureCount++;
                lastFailureTime = System.currentTimeMillis();
                log("call failed (" + failureCount + "/" + failureThreshold + "): " + e.getMessage());
                if (state == State.HALF_OPEN || failureCount >= failureThreshold) {
                    transitionTo(State.OPEN);
                }
                return fallback.call();
            }
        }

        private void transitionTo(State next) {
            if (state != next) {
                log("state " + state + " -> " + next);
                state = next;
            }
        }

        private void log(String msg) {
            System.out.printf("    [%s] %s%n", name, msg);
        }
    }
```

```
    // ---------- A service that fails for a while, then recovers ----------
    static class FlakyService {
        private final AtomicInteger calls = new AtomicInteger();
        private volatile boolean healthy = true;

        String getData() {
            calls.incrementAndGet();
            if (!healthy) {
                throw new RuntimeException("service unavailable");
            }
            return "real-data";
        }

        void breakIt()  { healthy = false; }
        void healIt()   { healthy = true; }
        int callCount() { return calls.get(); }
    }

    public static void main(String[] args) throws Exception {
        FlakyService service = new FlakyService();
        CircuitBreaker cb = new CircuitBreaker("payments", 3, 2000); // 3 failures, 2s open

        System.out.println("\n--- Phase 1: service healthy ---");
        for (int i = 0; i < 3; i++) {
            System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        }

        System.out.println("\n--- Phase 2: service breaks ---");
        service.breakIt();
        for (int i = 0; i < 6; i++) {
            System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        }
        System.out.println("  >> service was actually called "
                + service.callCount() + " times so far");

        System.out.println("\n--- Phase 3: still broken, circuit stays open ---");
        int before = service.callCount();
        for (int i = 0; i < 3; i++) {
            cb.call(service::getData, () -> "CACHED");
        }
        System.out.println("  >> extra real calls during OPEN: "
                + (service.callCount() - before) + "  (should be 0)");

        System.out.println("\n--- Phase 4: wait for timeout, service recovers ---");
        Thread.sleep(2100);
        service.healIt();
        System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
        System.out.println("  result = " + cb.call(service::getData, () -> "CACHED"));
    }
}
```

B. What you should see

```
--- Phase 1: service healthy ---
  result = real-data          (x3, circuit stays CLOSED)

--- Phase 2: service breaks ---
    [payments] call failed (1/3): service unavailable
    [payments] call failed (2/3): service unavailable
    [payments] call failed (3/3): service unavailable
    [payments] state CLOSED -> OPEN
    [payments] REJECTED (circuit OPEN) -> fallback
  >> service was actually called 6 times so far

--- Phase 3: still broken, circuit stays open ---
  >> extra real calls during OPEN: 0

--- Phase 4: wait for timeout, service recovers ---
    [payments] state OPEN -> HALF_OPEN
    [payments] probe SUCCEEDED
    [payments] state HALF_OPEN -> CLOSED
  result = real-data
```

The line that proves the pattern works is `extra real calls during OPEN: 0`. Three requests came in; the failing service received none of them. That's both benefits at once — the caller didn't wait, and the sick service got left alone.

C. Production — Resilience4j

Nobody hand-writes breakers in production. **Resilience4j** is the current standard for Java (Hystrix is in maintenance mode).

Dependency (Maven):

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.2.0</version>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Configuration (`application.yml` **)** — this is where the real learning is:

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        # Use a percentage over a rolling window, not a raw count
        slidingWindowType: COUNT_BASED
        slidingWindowSize: 20              # consider the last 20 calls
        minimumNumberOfCalls: 10           # don't judge until we have 10 samples
        failureRateThreshold: 50           # open if >50% of them failed
        waitDurationInOpenState: 10s       # how long to stay OPEN
        permittedNumberOfCallsInHalfOpenState: 3
        automaticTransitionFromOpenToHalfOpenEnabled: true
        # Treat slow calls as failures too - crucial, see pitfall 3
        slowCallRateThreshold: 50
        slowCallDurationThreshold: 2s
        recordExceptions:
          - java.io.IOException
          - java.util.concurrent.TimeoutException
        ignoreExceptions:
          - com.example.ValidationException   # never trip on a 400
```

Usage:

```
@Service
public class PaymentClient {

    @CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
    public PaymentResult charge(Order order) {
        return restClient.post()
                .uri("/payments")
                .body(order)
                .retrieve()
                .body(PaymentResult.class);
    }

    // Signature must match, plus a trailing Throwable
    private PaymentResult paymentFallback(Order order, Throwable t) {
        log.warn("Payment circuit open for order {}: {}", order.getId(), t.toString());
        return PaymentResult.deferred(order.getId());   // queue for later
    }
}
```

Monitoring – expose breaker state so you can alert on it:

```
management:
  endpoints.web.exposure.include: health,circuitbreakers,prometheus
  health.circuitbreakers.enabled: true
```

Then alert on the Prometheus metric `resilience4j_circuitbreaker_state`.

D. Implementation pitfalls

1. Using a raw failure count instead of a rate over a window. Three failures scattered across an hour of healthy traffic shouldn't trip anything. Always use `slidingWindowSize` + `failureRateThreshold`, and set `minimumNumberOfCalls` so a single early failure at 100% rate doesn't open the circuit.

2. Counting business errors as failures. A 400 or a validation exception means the *request* was bad, not the *service*. Put those in `ignoreExceptions` or you'll trip the breaker on a client bug and take down a healthy dependency.

3. Forgetting `slowCallDurationThreshold`. A service returning 200 OK after 30 seconds never "fails," so a failure-only breaker never opens — while your threads are consumed exactly as badly as if it were down. Slow-call detection is what catches the most dangerous case.

4. No timeout on the underlying call. The breaker can only react to a call that *returns*. Always set connect and read timeouts on your HTTP client; otherwise a hung socket blocks forever and the breaker never sees a failure to count.

5. One breaker shared across dependencies. Name breakers per downstream service. A shared instance means the reviews service failing blocks your payment calls.

6. Fallbacks that call other services. Your fallback runs when things are already broken. It must be local and fast — cache, default, or queue. A fallback making a network call is a second failure waiting to happen.

7. Not alerting on state. Covered above but it's the one people skip: a breaker doing its job hides errors. Alert on the open transition or you'll find out from a customer.

Related patterns

- **Bulkhead** — isolates resource pools; the breaker limits *how long* you try, the bulkhead limits *how much* damage one failure can do
- **Retry** — handles transient faults; the breaker stops retries becoming an attack on a downed service
- **Service Discovery** — per-instance breakers across a discovered pool
- **API Gateway** — a natural place to configure breaker rules per backend route
- **Sidecar** — service meshes provide breakers at the infrastructure level, with no application code